

supervisor	
signature	

* You should write your answer in English. Otherwise, penalty may be applied.

- The three main benefits of data encapsulation are (a. _____), (b. _____), (c. _____).
- (same as (c)) means (d. _____) in implementation should not affect (e. _____) seen by clients of a class.
- (f. _____) is the degree to which each program module relies on each one of the other modules.
- (g. _____) is a measure of how strongly-related and focused the various responsibilities of a software module are. It is often desirable to minimize (same as (f)) and maximize (same as (g)).
- (h. _____) member function does not modify any data members of the object on which it is called.
- One of the formal definitions of (i. _____) is "the establishment and use of sound engineering principles in order to (j. _____) obtain software that is reliable and works efficiently".
- (k. _____) is a simple dummy routines with no behavior or with very limited behavior, and it can be a temporary substitute for yet-to-be-developed code.
- (l. _____) contains a boolean expression that you believe will be true when (same as (l)) executes. If the boolean expression is not true, the system will throw an error. By verifying that the boolean expression is indeed true, the (same as (l)) confirms your assumptions about the behavior of your program. Experience has shown that writing (same as (l)) while programming is one of the quickest and most effective ways to detect (m. _____).
- Software crisis is the term used to describe the impact of rapid increases in computer power and the (n. _____) of the problems. Software technology could not follow the (same as (n)).
- Using (o. _____) variables can be dangerous because it can make all modules (p. _____) on one another.
- structured programming $\approx$ (q. _____) $\approx$ (r. _____) $\approx$ functional decomposition ($\approx$ means "similar concepts")

4. (14 points) (1) What is inline function? explain with sufficient details.
()

(2) In which situations, can we get benefits of using inline function? List at least two situations?
(1:), (2:)

(3) What is the main benefit we can obtain by using inline function? Why? Explain.
(What benefit? :), (Why?)

(4) What is the possible disadvantage of using inline function? Especially when can we have such disadvantage?
(What disadvantage?), (Especially when?)

5. (9 points) Assume the member variable “open_files” in line 2 stores the number of open files (i.e. the number of **File** objects).

(1) What is the meaning of “static” in line 2 of the following code?

What difference is made by inserting “static” in front of “int open_files”? Explain the difference with full details.

()

(2) Why is “static” necessary for the variable “open_files” in line 2? Explain.

()

(3) What problem will occur if you do not use “static” in line 2? Why? Explain.

()

```
1: class File {
2:     static int open_files;      // “open_files” stores the number of “File” objects
3:     // other details omitted
4: public :
5:     File(const String& filename, const String& modes) { ++open_files; } // other details omitted
6:     ~File() { --open_files; } // other details omitted
7:     // other details omitted
8:};
```

6. (9 points) Following C++ code tries to compute and print the area of a rectangle. However, the code contains a wrong part.

(a) Which line in the code is wrong? Mark the wrong part, where compilation error will occur, using an arrow and a circle.

(b) What is wrong with the part of code you marked? (Why is it wrong?) Explain. ()

(c) Insert one line code in line 3 to make the code correct. ()

<pre>1: #include <iostream> 2: class Rectangle { 3: 4: public: 5: Rectangle(int w, int h) { width=w; height=h; } 6: private: 7: int width; 8: int height; 9: };</pre>	<pre>10: int get_area(Rectangle c) { 11: return c.width * c.height; 12: } 13: int main() { 14: Rectangle rect(5,6); 15: std::cout << get_area(rect) << "\n"; 16: return 0; 17: }</pre>
---	--

7. (16 points) Consider a C++ integer stack class **TIntStack** that is defined as follows. Read comments carefully in the code.

```
#include <iostream>
const unsigned int DEFAULT_SIZE = 256;
class TIntStack {
public :
    TIntStack(unsigned int stackSize = DEFAULT_SIZE);
    TIntStack(const TIntStack& that);
    ~TIntStack();
    bool Push(int thisValue); // return true if successful
                             // return false if stack is full
    unsigned int getCount() const { return _count; }
private :
    int* _sp; // memory is dynamically allocated
           // during constructor function call
    unsigned int _count; // number of elements in stack
    unsigned int _size; // size (capacity) of stack
};
```

(1) Write appropriate C++ code for following constructor.

TIntStack::TIntStack(unsigned int stackSize)

(2) Write appropriate C++ code

for following copy constructor that does deep copy.

TIntStack::TIntStack(const TIntStack& that)

(3) Write an appropriate C++ code for following destructor.

TIntStack::~TIntStack()

(4) Write an appropriate C++ code for member function **Push**

bool TIntStack::Push(int thisValue)